

Five changes I made to AstroLive, and why

Each one started from something I noticed — either using the app, reading about how these apps make money, or just watching what my friends actually do. This document explains the thinking first, then how it is built.

Built for mobile web (mweb) **Stack** React 19 · Vite · Firebase

Code github.com/Narasimhagupta2004/astrolive-hackthon

Team Bandaru Narasimhagupta (SDE1, Infinite Locus) · Karri Naveen (SDE1, Jungleworks)

Before you read the features

Two things worth knowing up front, because they shape everything below.

This is a mobile-web prototype, not the app. I built it as mweb because it is the fastest way to put a working thing in someone's hand — you send a link, it opens, no install. But I want to be clear that I know where the money actually is: **the app will always out-earn the web.** Push notifications, a home-screen icon, saved payment methods, and sessions that don't end when a tab closes — those are what drive repeat consultations, and the browser gives us weak versions of all four. So treat everything here as a design proven on mweb and meant to move to the app. Nothing in it is web-specific.

I could not see the astrologer's side of the product. I have only ever used AstroLive as a customer, so for the first feature I had to guess what an astrologer's panel looks like. I have marked that assumption clearly rather than hiding it.

Trying it yourself

Razorpay runs in **test mode**, so nothing is charged. Two places take a payment: Shubh Kart checkout, and AstroLive Plus on the Muhurat page. Use this card for both.

```
Card      4386 2894 0766 0153
Expiry    any future date      # e.g. 12 / 30
CVV       any 3 digits           # e.g. 123
OTP       any digits           # the test gateway accepts anything
```

No real card works in test mode and no real card should be entered. To reset the demo to a fresh user, clear `astro:coins` and `astro:subscription` from the browser's local storage.

What it is built with

Every tool in the project, and the one job it does.

React 19	→	UI
Vite	→	Build and dev server
Redux Toolkit	→	The coin ledger
React Context	→	Cart and session state
Firebase	→	Prototype backend — anonymous auth and live sync
localStorage	→	Client-side persistence
Google Gemini	→	Muhurat AI answers, with a local rules-engine fallback
Razorpay	→	Payment integration
lucide-react	→	Icons
GA4 via GTM	→	Analytics and funnels PLANNED
Clarity	→	User behaviour and session recordings PLANNED
GrowthBook	→	Feature flags and A/B experiments PLANNED

The three marked *planned* are not wired up yet — they are the measurement stack proposed in section 10. Everything above them is in the build today.

Why each one

Tool	Used for	Why this one
React 19	The whole interface	Standard, well understood, easy to hand over
Vite	Build and dev server	Instant reload, and it opens on a real phone over Wi-Fi while you work
Redux Toolkit	The coins ledger only	Coins turn into money, so they need stricter handling than the rest
React Context	Cart, recent sessions	Lighter than Redux, right for data that is short-lived

Tool	Used for	Why this one
Firestore	Live sync for Cosmic Chemistry	Real-time updates with no server to run
Google Gemini	Muhurat AI	Grounded on our own panchang, so it explains rather than invents. Optional — without a key the rules engine answers
Razorpay	Shubh Kart checkout and AstroLive Plus	Standard Indian gateway; loads only at checkout
lucide-react	Icons	One consistent set, ships only the icons used

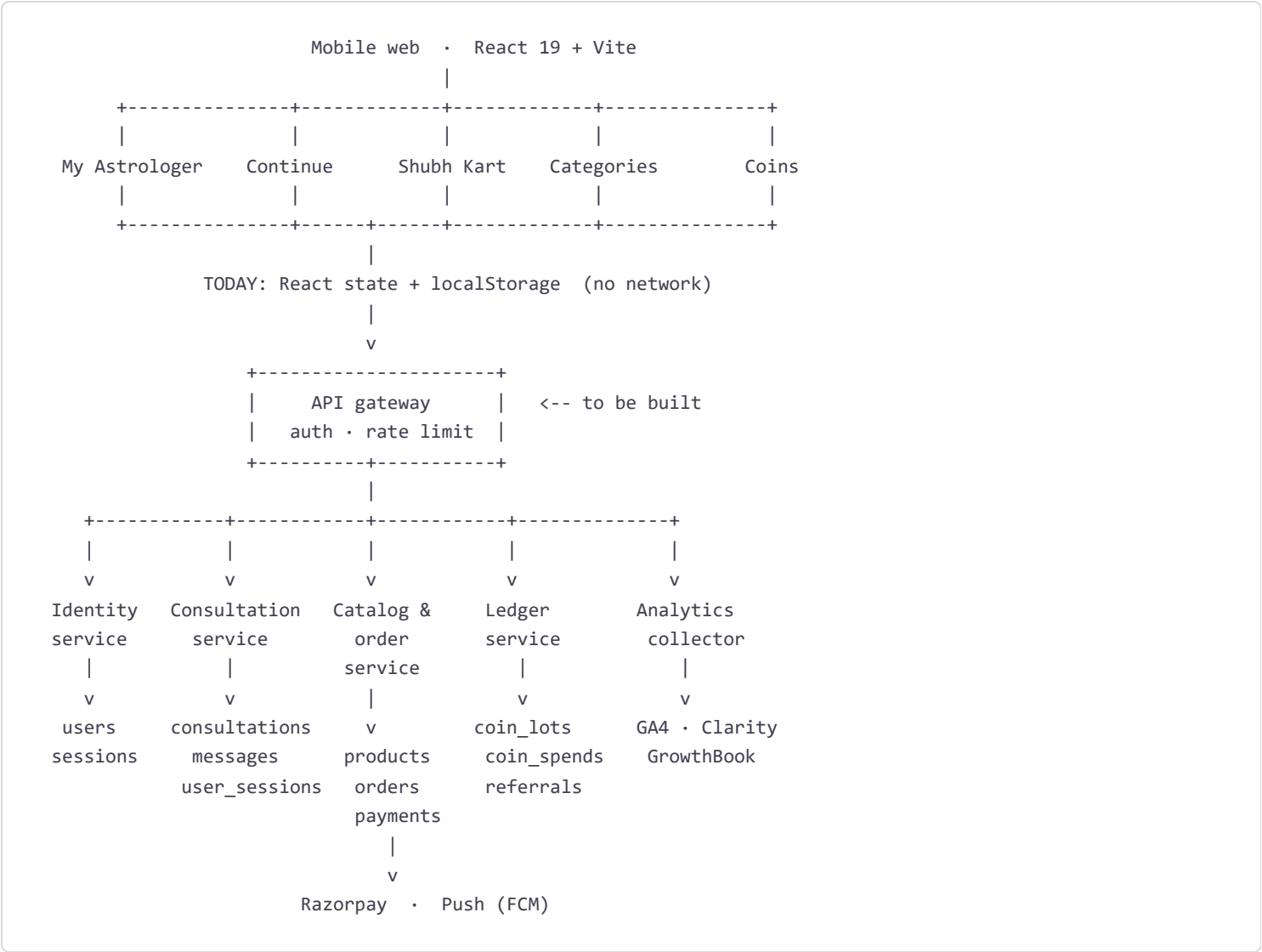
How the folders are laid out

```
src/
├─ pages/      one file per screen – 21 screens
├─ components/ reusable pieces, grouped by area
├─ data/       every business rule and price, in one place
├─ state/     shared memory: cart, sessions
├─ store/     the coins ledger
├─ utils/     anything that talks outside (Firestore, Razorpay)
└─ styles/    one stylesheet
```

One rule holds it together: **pages arrange, components display, data decides, state remembers, utils connect**. Every cap, price and rule sits in `data/` rather than buried inside a screen — so changing "3 connects" to "5 connects" is one line in one file, not a hunt through twenty.

System architecture

One picture for the whole product, so the five features below can be read as parts of a system rather than five separate screens. Solid lines exist today; the API tier and the services under it are what we build next.



What each layer owns

Layer	Owns	Today
Screens (pages/)	Layout and navigation only	Built
Components	Rendering; no business rules, no network	Built

Layer	Owns	Today
State (<code>state/</code> , <code>store/</code>)	Cart, sessions, coin ledger	In browser memory + localStorage
Rules (<code>data/</code>)	Caps, prices, eligibility — pure functions	Built, and reusable by the server unchanged
Adapters (<code>utils/</code>)	The only files that talk outward	Firebase + Razorpay only
API tier	Auth, validation, money, quotas	Not built
Data tier	Durable records	Not built

That fourth row is the one that makes the migration cheap. Because every rule is already a pure function in `data/` — `maxRedeemable()` , `checkCredit()` , `MAX_CONNECTS` , `FOLLOWUP_CALL_SECS` — the server can import the same file and enforce identical numbers. There is no second copy of the rules to drift out of sync.

One request, before and after

```
# today — nothing leaves the browser
tap Add -> CartContext.add(productId) -> useMemo recomputes totals -> re-render

# after the API tier exists
tap Add -> POST /v1/cart/items {productId, qty}
        -> gateway: verify JWT, rate limit
        -> catalog service: check stock and price
        -> DB: upsert cart_items
        <- 200 {cart: {items, subtotal, giftUnlocked}}
        -> client renders the server total, never its own
```

The client stops being the source of truth for anything that costs money. It keeps deriving display values, but the number it shows comes back from the server.

How to read each feature below

Every feature is written to the same seven-layer strip, so they can be compared directly and picked up as work items: **problem** → **user flow** → **frontend** → **API** → **data** → **events** → **metric**.

My Astrologer

ASSUMPTION — FLAGGED, NOT HIDDEN

I have never seen the astrologer panel. I have only used AstroLive as a customer, so everything in this section is a proposal for what should sit inside that panel, not a description of something that exists. The reasoning and the limits below stand on their own; if the panel already works differently, this needs re-placing rather than re-thinking.

This is the one part of the document that needs a correction from someone on the supply side before it is built.

A consultation happens once. The user asks, the astrologer answers, the session ends, and then nothing. There is no follow-up at all — so every relationship starts again from zero.

What I built

A view of the app from the astrologer's side. Turn on **Astrologer Mode** and the menu gains **My Users** — the people they have already consulted — with three follow-up actions: pick the chat back up, ring them, or suggest a remedy from Shubh Kart.

Why it works

A message from the person who just read your chart is the most persuasive thing in the app, and it costs nothing to send. It also makes the user feel remembered, which is what actually brings people back.

The obvious risk is that it turns into spam. So this is a **quota, not a broadcast**: three connects per user, ever. The astrologer sees "2 of 3 connects left" before spending one, which pushes them to use it on the users who genuinely matter. And because every follow-up is recorded, nobody can quietly abuse it.

The follow-up call is capped at 2 minutes

A follow-up call has to be a nudge, not a free reading. Two minutes is long enough to say "I've been thinking about your chart, shall we book a proper session" and not long enough to give away a consultation. It protects paid minutes while still feeling personal, and it makes the intent unambiguous on both sides — the astrologer knows it is an opener, and the user knows they are not about to be sold to for twenty minutes.

The timer runs on the call screen and ends the call at 2:00, the same way the connect counter works: the limit lives in the data, not just in the button.

Engineering HLD

USER FLOW	Settings toggle → My Users → Connect → sheet → chat / call (2 min) / suggest → counter decremented
FRONTEND	<code>ConnectedUsersPage</code> , <code>CallPage</code> , <code>SuggestProductPage</code> , <code>BottomSheet</code> ; quota in <code>SessionContext</code> , cap in <code>data/appData.js</code>
API	<code>GET /astrologer/me/users</code> · <code>POST /astrologer/connects</code> · <code>POST /calls</code> · <code>POST /suggestions</code>
DATA	<code>consultations</code> , <code>astrologer_connects</code> , <code>calls</code> , <code>product_suggestions</code>
EXTERNAL	Telephony provider for the call, push for the follow-up message
EVENTS	<code>astrologer_connect_used</code> , <code>followup_call_ended{duration_sec, capped}</code> , <code>product_suggested</code>
METRIC	Paid sessions booked per follow-up, against block/report rate

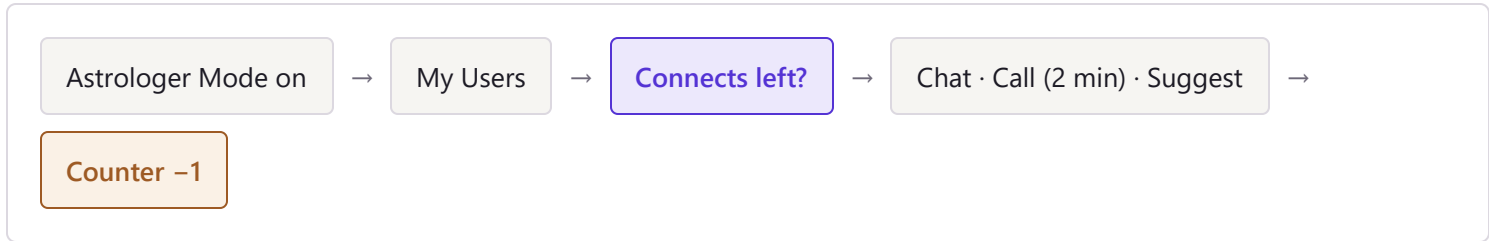
```
GET /v1/astrologer/me/users?cursor=&limit=20
200 { users: [ { userId, name, rashi, lastMode, lastAt,
                consultations, connectsUsed, connectsLeft } ], next }

POST /v1/astrologer/connects
  { userId, mode: "chat" | "call" | "suggest" }
200 { connectsLeft: 2 }
409 { error: "connect_limit_reached" }      # server owns the cap, not the button

POST /v1/calls
  { userId, kind: "followup" }
200 { callId, maxDurationSec: 120 }        # server returns the cap it will enforce

POST /v1/suggestions
```

```
{ userId, productId }  
200 { suggestionId } # becomes the attribution key for commission
```



How it is built

- One on/off setting swaps a tab into the menu — no duplicate screens, no second app.
- The three actions run off a single lookup table, so a fourth action is one line.
- The counter and the call cap both live in stored data, so the rules hold even if a button is missed.
- A suggested product arrives *as a message, not an order*. The user can ignore it, which is exactly why they trust it.

Continue where you left

Every visit starts at a grid of astrologers. So people pick a different one each time, get the same question answered three different ways, and end up more confused than when they arrived.

WHERE THIS CAME FROM

In real life nobody shops around like this. You go back to the same astrologer, year after year, because they already know your situation. The app was doing the opposite of what people actually do — and when three readings contradict each other, the user doesn't conclude that one of them was wrong. They conclude the whole thing is nonsense, and they leave.

What I built

The home screen now opens with the last astrologers you spoke to — chat or call, how long ago, and a **Resume** button.

Why it works

Continuity is what makes advice feel credible. Putting the last person you spoke to at the top makes going back the easy choice and starting over the deliberate one.

And trust travels. Once someone has stayed with the same astrologer long enough to believe in them, they stop recommending the app and start recommending the person — "go to this astrologer, they were right about my job." That is a much stronger referral than a generic invite, because the new user arrives already knowing who to book. It also gives us something to build on: a share card for a specific astrologer, sent by someone who has actually consulted them.

The list stops at **two** on purpose. A resume list exists to remove a decision. A list of ten just recreates the paralysis it was built to fix.

Engineering HLD

USER FLOW	Start a chat → session saved → return to Home → Resume at the top
FRONTEND	<code>ResumeSection</code> on <code>HomePage</code> ; <code>startSession()</code> in <code>SessionContext</code> , LRU capped at 2
API	<code>GET /me/sessions?limit=2</code> · <code>POST /me/sessions</code> (upsert on chat or call start)
DATA	<code>user_sessions(user_id, astrologer_id, mode, started_at, thread_id)</code> , unique on (user_id, astrologer_id)
EXTERNAL	None
EVENTS	<code>resume_card_shown</code> , <code>resume_card_tapped{astrologer_id, age_minutes}</code>
METRIC	Repeat consultations with the same astrologer, and referrals naming an astrologer

```
GET /v1/me/sessions?limit=2
200 { sessions: [ { astrologerId, mode, startedAt, threadId } ] }
# ids only – the client joins against the live astrologer record,
# so a stale rate or offline dot can never be served from cache

POST /v1/me/sessions
  { astrologerId, mode, threadId }
200 { ok: true }
# upsert: same astrologer moves to the top instead of duplicating
```

User starts a chat → **Saved on the device** → Comes back later → Home shows Resume

How it is built

- Only the astrologer's ID and the time are saved. The live profile is fetched fresh every time, so the rate and the online dot are never stale.
- Newest first, duplicates removed, trimmed to two.
- Saving is wrapped safely so private-browsing mode degrades quietly instead of breaking the page.

Next

Hold the history on the account so it follows the user to a new phone, and reopen the actual conversation rather than the profile.

Shubh Kart recommendations

The store is a flat grid with no recommendations at all. Someone who has just been told they have Shani dosh has to work out for themselves which of nine products relates to that.

WHERE THIS CAME FROM

I went through a few competitor apps and some write-ups on how this category makes money, and the pattern is consistent: a large share of the revenue comes from suggesting the right product at the right moment, not from the consultation itself. We already have the strongest possible salesperson — the astrologer the user just paid to trust — and we were not using them.

What I built

Two ways to reach the right product, for two different moments:

- **The user filters by intent** — Peace, Love, Career, Wealth, Protection, Growth — and picks their rashi to see suitable items first, marked *For You*.
- **The astrologer suggests directly**. After a reading they can send one specific remedy to that user.

Why it works

The second path is the valuable one, and it costs nothing to run. One design decision is worth calling out: **intent narrows the list, but rashi only reorders it**. If rashi filtered too, most of the catalogue would disappear and browsing would die. Personalisation should reorder a shop, never shrink it.

Engineering HLD

USER FLOW

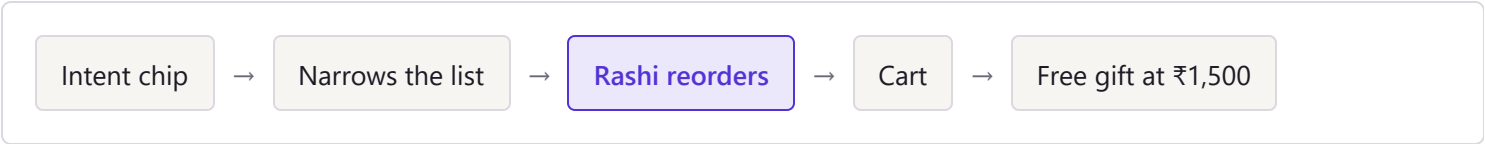
Intent chip narrows → rashi reorders → product sheet → cart → address → pay. Or: astrologer suggests → same cart

FRONTEND	ShubhKartPage , IntentChips , RashiPicker , ProductCard/Sheet ; CartContext derives every total
API	GET /products · POST /cart/items · POST /orders · POST /orders/:id/verify
DATA	products , product_targeting , carts , cart_items , orders , payments
EXTERNAL	Razorpay — order creation and signature verification, both server-side
EVENTS	product_viewed , add_to_cart , order_placed , suggestion_purchased{suggestion_id}
METRIC	Shubh Kart revenue per consultation; attach rate on astrologer suggestions

```
GET /v1/products?intent=peace&rashi=mesha
200 { products: [ { id, name, price, oldPrice, rating, forYou: true } ] }
# intent filters server-side, rashi only reorders – never hide the catalogue

POST /v1/orders
  { addressId, deliveryDay, suggestionId? } # suggestionId carries attribution
200 { orderId: "SK-...", razorpayOrderId, amount }

POST /v1/orders/:orderId/verify
  { razorpayPaymentId, razorpaySignature }
200 { status: "paid" }
# HMAC checked with the Razorpay SECRET, which never reaches the client.
# Without this step a client can claim any payment succeeded.
```



The astrologer's suggestion feeds the same cart from the other direction.

How it is built

- Each product carries its own targeting — which intents and rashis it suits — so merchandising is a data change, not a code change.
- Totals, item count and the free-gift progress bar are always recalculated from the cart, so a total can never disagree with its items.
- Razorpay only loads at checkout, and switches off cleanly if no key is configured.

If a product we don't stock is one the astrologer has, we can list it in Shubh Kart after verifying it, and pay the astrologer a commission. Two things have to exist first: a verification step for authenticity, stock and returns, and attribution — recording which astrologer suggested the sale — because you cannot split a commission you cannot trace. Beyond that, ranking should use real signals: what the consultation was about, past purchases, and the user's kundli.

Categories by problem, and Life Coach

The old tabs were All · Vedic · Tarot · Numerology. Nobody opens the app thinking "I need Numerology today." They open it thinking "will I get this job." We were labelling our method when the user is buying an outcome.

WHERE THIS CAME FROM

I messaged some friends asking where they were. They said they'd gone to a yoga session and to hear a talk from a Vedic priest. I found that genuinely funny — these are people who never listened to a single word their parents said, and now they're sitting through this voluntarily. It's a good thing, though. Somewhere in an hour like that, one point lands and actually changes something. That's when it struck me: the appetite here is for *guidance*, not specifically for astrology. So why aren't we offering life coaching properly?

What I built

Categories are now the problems people actually have, and Life Coach is one of them:

Category	What the user is really asking
Auspicious Timing	When should I do this?
Love & Marriage	Is this the right person?
Career & Money	Will this job or investment work out?
Health & Peace	Why do I feel like this?
Life Coach	What should I do with my life?

Why it works

A user reads the tab bar and immediately finds themselves in it. That is the entire job of a category list, and the old one wasn't doing it.

Life Coach also opens a market with no religious prerequisite, which is a much wider funnel — and we already have the supply, because several astrologers already counsel this way.

It is the one category that suits a **subscription**. Astrology is episodic: you consult when something is wrong. Coaching is continuous. Give one or two sessions free for a month, let the habit form, then charge monthly — that matches how people actually use it.

Engineering HLD

USER FLOW	Open Chat → pick a problem tab → filtered astrologers → chat. Empty state offers a reset
FRONTEND	<code>ChatTabs</code> (controlled), filter in <code>ChatPage</code> , <code>categoryChips()</code> on <code>AdvisorCard</code>
API	<code>GET /categories</code> · <code>GET /astrologers?category=&sort=</code> · <code>PUT /astrologers/:id/categories</code> (verified writes only)
DATA	<code>categories</code> , <code>astrologer_categories(astrologer_id, category_id, source, confidence)</code> — max 3 rows per astrologer
EXTERNAL	None
EVENTS	<code>category_selected{category_id, results_count}</code> , <code>category_empty_state_shown</code>
METRIC	Browse-to-chat conversion, and time to first chat

```
GET /v1/astrologers?category=coach&sort=rating&online=true
200 { astrologers: [ { id, name, skills[], categories[], ratePerMin,
                      rating, isOnline, isLive } ] }

PUT /v1/astrologers/:id/categories # ops / verified writes only
  { categories: ["career", "coach"], source: "verified" }
200 { ok: true }
422 { error: "too_many_categories", max: 3 }
# the cap is what stops every astrologer claiming every category
# and turning the filter back into decoration
```

Skills: Vedic · Tarot



Problems solved: Career · Life Coach



Listed under those tabs

How it is built

- An astrologer keeps their *skills* (how they work) and gains *categories* (what they solve). One astrologer can serve several problems.
- The category list is defined once and drives the tabs, the card labels and search together — a new category is one line.
- Each category has a long name for tabs and a short one for cards, so nothing gets cut off on a small screen.
- If a category has nobody free, the user gets a clear message and a one-tap way back to everyone.

Next — the part that needs care

The hard question is **who decides an astrologer's categories**. Get it wrong and every astrologer claims every category, and the filter is useless again. The plan: astrologers self-declare at onboarding, we check that against what their consultations were actually about, and cap each one at three categories. For the Life Coach subscription we also need trial and renewal tracking, which is a backend piece.

Referrals, sharing and coins

Paid installs are expensive. The cheapest real growth is one user telling another — but only if there is a reason to. And the trap on the other side is just as real: reward sharing carelessly and you end up funding fraud instead of growth.

WHERE THIS CAME FROM

In college my friends were forever passing around referral links to make a bit of money. I once put in a solid fifty minutes of effort for a single signup — and yes, there was a shorter route from my hostel to theirs, but let's not get into that. The point survives the joke: people will go to genuinely absurd lengths for a small reward, as long as the reward is real and they can see it. That is a growth channel you cannot buy.

What I built

Users share a Cosmic Chemistry reading with a friend and earn coins, which come off their next wallet recharge.

Rule	Value
Coin value	10 coins = ₹1
A reading completed with someone new	+5 coins · max 3/day · 100 lifetime
A referred friend's first recharge of ₹200+	+500 coins (₹50)
Coins can cover	up to 10% of a recharge of ₹100+
Coins expire	90 days, oldest used first

Why this cannot lose the business money

The 10% cap is the whole defence. To use ₹50 of coins, a user has to recharge ₹500. Every coin redeemed is attached to revenue at least ten times bigger, so the programme funds itself by design — that is arithmetic, not optimism.

Coins held	Worth	Recharge needed to use them all
200	₹20	₹200
500 (one referral)	₹50	₹500
1,000	₹100	₹1,000

Abuse is capped as well. The share bonus stops at 100 coins for life — ₹10 per person, once, ever — and even that only converts against ₹100 of spend. Gaming it costs more than it pays.

Engineering HLD

USER FLOW	Share a reading → friend completes it → coins credited → recharge → up to 10% paid in coins
FRONTEND	CosmicSharePage , WalletPage , RechargePage ; ledger in store/coinsSlice.js , rules in data/coins.js
API	GET /me/coins · POST /recharge/quote · POST /recharge . Awarding is server-internal, never client-callable
DATA	coin_lots , coin_spends , coin_dedupe , referrals , recharges
EXTERNAL	Razorpay for the recharge payment; a daily job expires lots past 90 days
EVENTS	share_completed , coins_awarded{source} , coins_redeemed{coins, recharge_amount}
METRIC	Referred installs, and recharge frequency against margin per recharge

```
GET /v1/me/coins
200 { balance: 240, expiringSoon: 15, history: [ ... ] }
# read-only. The client can never write a balance.

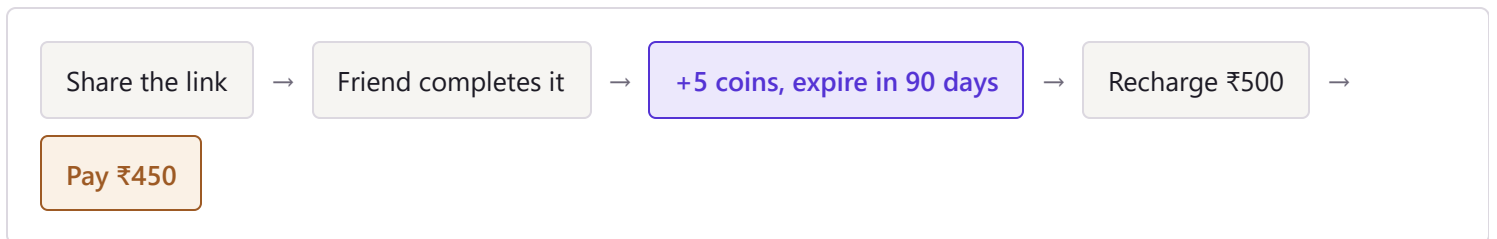
POST /v1/recharge/quote
  { amount: 500 }
200 { maxRedeemableCoins: 500, discount: 50, payable: 450 }
# the 10% cap is recomputed server-side from the amount, every time
```

```

POST /v1/recharge
  { amount: 500, useCoins: true }
  Idempotency-Key: <uuid>           # a retry must not double-charge
200 { rechargeId, razorpayOrderId, coinsApplied: 500 }
# coins are burned in the SAME transaction as the payment capture,
# so a failed payment cannot silently eat the balance

# awarding — internal only, triggered by a verified event
onRevealComplete(roomCode):
  read room server-side, require status == "complete"
  require b.uid != a.uid           # no self-referral
  require not exists coin_dedupe(uid, partnerUid)
  apply dailyCap 3, lifetimeCap 100 # same checkCredit() the UI uses
  insert coin_lot(+5, expiresAt = now + 90d)

```



How it is built

- **Every coin batch is tracked separately**, each with its own expiry date — so coins can expire, be audited, or be reversed. A single running total could do none of those.
- **Coins nearest expiry are spent first**, so users never lose value they could have used.
- **The same eligibility check runs twice** — once when the button is drawn, once when the coins are written — so the screen and the ledger can never disagree.
- **Every reward type is a config entry** with its own caps, so a festival campaign is a settings change, not new code.
- **The 10% cap is recalculated live** from the selected amount, so it can never fall out of step.
- The rules and the customer-facing terms come from **one file**, so what we enforce and what we promise stay identical.

Next — required before this touches real money

Coins become a discount on a payment, so the balance has to be held and awarded on a server, not on the phone. That means phone-number sign-in, a server-side ledger, and awards tied to a verified recharge. Until that exists my recommendation is to show coins in the profile but not let them reduce a payment. The full write-up is in `docs/coins-rewards-plan.md` .

Muhurat AI

"When should I do this?" is the single most common question in the category, and answering it properly means reading a panchang — which almost nobody can do. So people either guess, or pay for a full consultation to get one date.

What I built

Pick what you are planning — signing a contract, buying property, a wedding — and get three auspicious windows in the next 14 days, each with the nakshatra, the hora, the tithi, and the Rahu Kaal it avoids. Answers arrive in seconds instead of a booked call.

The first reading is free and credits 25 coins. The second asks for **AstroLive Plus** — ₹99 for 30 days, which also credits 100 coins. One payment, one term, no auto-renewal.

Why it works

This is the cheapest possible first taste of the product. It answers a real question with no signup, no wallet balance, and no waiting for an astrologer to come online — and it ends with coins sitting in a wallet that are only worth anything on a recharge. The free reading is a demonstration that pays for its own follow-up.

The paywall is placed where intent is highest: the user has already had one useful answer and is typing a second question. That is a far better moment to ask for ₹99 than a banner on the home screen.

Both rewards are attached to something: the free 25 coins to a completed reading, the 100 coins to a payment we already took. Neither can be farmed — the free one is deduped once per account, the paid one on the payment id.

Engineering HLD

USER FLOW

Pick intent → free reading → 25 coins → second ask → paywall → pay ₹99 → 30 days + 100 coins

FRONTEND	MuhuratPage , IntentGrid , MuhuratCard ; gate is freeMuhuratUsed && !isSubscribed , both read from Redux
API	POST /muhurats · GET /me/entitlements · POST /subscriptions
DATA	muhurat_requests , subscriptions , coin_lots , plus the panchang corpus
EXTERNAL	Google Gemini, grounded on our panchang JSON; Razorpay for the plan
EVENTS	muhurat_requested{intent} , muhurat_paywall_shown , plan_purchased{price}
METRIC	Free-to-paid conversion, and recharge rate among users holding muhurat coins

```
POST /v1/muhurats
  { intent: "new_job", intentText?, rashi: "mesha" }
200 { windows: [ { date, from, to, nakshatra, hora, tithi,
                  avoidsRahuKaal, why } ], poweredBy }
402 { error: "quota_exhausted", plan: { price: 99, days: 30 } }
# the free-ask quota is counted server-side – the client cannot grant itself one

POST /v1/subscriptions
  { plan: "plus", razorpayPaymentId }
  Idempotency-Key: <payment id>
200 { expiresAt, coinsAwarded: 100 }
# term and coin bonus commit together, keyed on the payment, so a replayed
# call extends nothing and credits nothing twice
```

How the AI is kept honest today

The model is never asked to know astrology. We compute a panchang table ourselves and pass it in as JSON; Gemini may only select and explain windows that already exist in that table, and the response shape is validated before it renders. If the call fails, times out, or comes back malformed, the local rules engine answers instead and the footer says so. A missing API key is just another fallback path, not an outage.

Next — a RAG model over verified sources

Grounding on our own table is safe but narrow: the answer can only ever be as good as the panchang we generated. The next step is retrieval over sources an astrologer would actually accept — digitised panchang almanacs, the classical muhurta texts (*Muhurta Chintamani*, *Kalaprakasika*, *Brihat Samhita*), and our own senior astrologers' recorded rulings.

Retrieve the relevant passages first, then let the model answer **only** from what was retrieved, citing which text each window came from. Two things change: an astrologer can audit an answer instead of trusting it, and "Thursday 06:00–07:30, per Muhurta Chintamani on Guru hora" is a materially different product from an unsourced date. It also fixes freshness — a new panchang year becomes a re-index, not a prompt rewrite.

The guardrail stays either way: the model may never emit a date that was not in the retrieved set, and anything that fails validation falls back to the rules engine rather than reaching the user.

How we would know if any of this worked

Every feature above is a hypothesis. None of them should stay in the product just because they sound reasonable — including mine. Here is how I would measure them.

Run them as experiments, not launches

GrowthBook for A/B testing: it is open-source, self-hostable, and does feature flags and experiments in one place — so a feature can be switched off without a release. Each of the five has an obvious test:

Experiment	What we would measure
Resume section on / off	Repeat consultations per user, and whether they go back to the same astrologer
Problem categories vs. old skill tabs	Browse-to-chat conversion, and time to first chat
Connect quota of 3 vs. 5	Follow-up conversion against block and report rate — the point where helpful turns annoying
Follow-up call at 2 min vs. 5 min	How many short calls convert into a paid session
Astrologer suggestion on / off	Shubh Kart revenue per consultation
Coin cap at 10% vs. 15%	Recharge frequency against margin per recharge

Watch what people actually do

- **Microsoft Clarity** — session recordings and heatmaps. This is the one that answers "why did nobody tap that": rage clicks, dead clicks, where people stop scrolling. It is free, and it catches things a funnel chart never will.
- **GA4, deployed through GTM** — the funnel numbers. GTM matters here because we can add or fix a tag without waiting for a release, which is the

difference between measuring a launch and measuring it three weeks late.

- **Segment** as a CDP, if and when we need it. Not on day one. It earns its cost once several tools all need the same clean event stream and we are tired of instrumenting each one separately.

The events I would define first

`resume_card_tapped` · `category_selected` · `astrologer_connect_used` · `followup_call_ended` (with duration, so we can see whether 2 minutes is the right cap) · `product_suggested` · `suggestion_purchased` · `share_completed` · `coins_redeemed` . Six of those eight are things we currently cannot see at all.

11

What moves to a backend, in order

#	Feature	What it needs
1	Everything	Phone-number sign-in — unblocks all of the below
2	Coins	Server-held balance, awarding and redemption
3	Shubh Kart	Order creation and payment verification on the server
4	My Astrologer	Real consultation records, server-side connect quota and call cap
5	Continue where you left	History on the account so it survives a new phone
6	Categories	Verified category assignment on astrologer profiles
7	Muhurat AI	Server-side free-ask quota and plan entitlement; the Gemini key moves off the client

The code was written with this move in mind. The coins ledger in particular has one clearly marked place where saving happens, so switching it to a server is a swap rather than a rewrite.

How an SDE1 picks this up

Everything above is designed to be built in slices, by one engineer, without a rewrite between slices. This is the order and the shape of the work.

Build order — each milestone ships on its own

#	Milestone	Done when
M0	Phone OTP auth + a <code>users</code> table; issue and verify JWTs at the gateway	Every later endpoint can identify the caller. Nothing else starts before this
M1	Coin ledger service — <code>coin_lots</code> , <code>coin_spends</code> , <code>coin_dedupe</code> , expiry job	A client write to a balance is rejected; caps and dedupe hold under a replayed request
M2	Recharge with transactional redeem + idempotency key	A forced payment failure mid-redeem leaves the balance untouched
M3	Catalog and orders; move Razorpay order creation and signature verify server-side	A forged payment callback is rejected
M4	Consultation service — history, connect quota, 2-minute call cap	A 4th connect returns <code>409</code> ; a follow-up call is cut at 120s server-side
M5	Astrologer profiles with verified categories, max 3	A 4th category returns <code>422</code>
M6	Analytics: GTM container, GA4 events, Clarity, GrowthBook SDK	All eight events fire with correct properties; one flag can dark-launch a feature

Slicing a milestone into tickets

Each milestone breaks the same way, and every ticket should be a day or less:

1. **Schema + migration** — tables, indexes, constraints. Put the rule in a constraint wherever the database can hold it (unique on `coin_dedupe`, max-3 check on categories) so a bug in the service cannot corrupt data.

2. **Service function with the rule** — import the existing pure function from `data/`. Do not rewrite `maxRedeemable()` or `checkCredit()` on the server; that is how the two copies drift apart.
3. **Endpoint** — validate input, authorise, call the service, map errors to status codes.
4. **Swap the client** — replace one context or slice with a fetch. The component tree should not change; if it does, the boundary was drawn in the wrong place.
5. **Events** — fire the analytics event from the same place the state changes, not from a click handler, so it cannot report a success that did not happen.
6. **Tests** — the abuse cases, not the happy path. Self-referral, replayed award, 4th connect, payment failure mid-redeem, ₹99 recharge.

Where to change the client

Replace this	With
<code>store/index.js</code> <code>localStorage</code> subscriber	Sync against the ledger API. It is already commented as the seam
<code>SessionContext</code> reads	<code>GET /me/sessions</code> , <code>GET /astrologer/me/users</code>
<code>CartContext</code> local totals	Server cart; keep the <code>useMemo</code> for display only
Static arrays in <code>data/appData.js</code>	<code>GET /products</code> , <code>GET /astrologers</code>
Route string switch in <code>App.jsx</code>	<code>react-router</code> , once URLs matter for sharing and analytics

`data/coins.js` and the caps in `data/appData.js` are the exception — they do not get replaced. They get imported by the server.

Conventions to keep

- Rules live in `data/` , as pure functions. If a rule needs app state to run, it is in the wrong file.
- Persist ids, join at read time. Never cache a whole product or astrologer object.
- Derive, never store. Totals, caps, counters are all recomputed. Two copies of a number are two chances to be wrong.

- **Registries over branches.** A new coin source, connect action, or category is a data entry.
- **The server owns anything that costs money or limits a user.** The client may display a cap; it must never be the thing enforcing it.

Team

Bandaru Narasimhagupta — SDE1, Infinite Locus

Karri Naveen — SDE1, Jungleworks

Code: github.com/Narasimhagupta2004/astrolive-hackthon